

分布式任务调度Crane架构与原理

🔍 | 📄 正文 4890 字 (7.1 页) | 🌐 浏览 0 次 (0 人) | 🏠 全部 >

AI 速览

Crane是一款分布式任务调度中间件，支持精确到秒的定时任务调度，适用于视频转码、日志清理、数据迁移等多样化场景。它通过分布式架构解决传统定时任务的单点问题，任务可在多台客户端机器上自动容灾切换。调度端和客户端均为分布式设计，高可用特性体现在弱依赖ZK和数据库，即使组件宕机也能维持核心。用户调度能力只需引入客户端jar包并添加注解即可快速接入，轻量服务且简单易用。整体架构分为scheduler（调度器）、client（业务方）、manager（集群管理）、monitor（心跳检测）四大模块，利用MySQL存储任务元数据，通过延迟队列实现周期调度，并采用slot分片机制支持水平扩展。manager通过租约机制和时间桶实现心跳管理与脑裂防护，确保调度器高可用。任务执行支持拥塞控制、自动重试、超时管理，并通过ZK实现节点注册与发现，保障系统稳定可靠。

Crane如何通过分布式架构解决传统定时任务的单点问题？

Crane的manager模块如何通过租约机制和时间桶实现心跳管理与脑裂防护？

🎨 风格切换 ▾

💬 继续提问



📖 本文crane架构内容基于crane 1.3.3版本，源码更新时间2019-12-01

简介

分布式任务调度Crane是一款任务调度中间件。支持分布式定时任务调度，支持国际通用的时间表达式（精确到秒），支持任务执行生命周期管理和历史执行记录查询，解决传统定时任务单点问题。

分布式任务调度Crane是一款任务调度中间件。支持分布式定时任务调度，支持国际通用的时间表达式（精确到秒），支持任务执行生命周期管理和历史执行记录查询，解决传统定时任务单点问题。可以用在比如每秒钟进行一次视频转码，每小时清理一遍日志文件，每天进行的历史数据迁移，每周五生成报表并发出邮件，每个月的最后一天发工资，每年一次的生日提醒等等场景，Crane都能轻松为您提供任务调度服务。

 分布式	不管是Crane调度端还是客户端（业务机器）都是分布式架构设计，任务可以在多台客户端机器里的任何一台机器执行，如果客户端出现宕机的情况，调度端会自动选择存活的客户端（业务机器）去执行任务。
 高可用	调度端按业务BG隔离。各个BG之间的任务调度互不影响。调度端依赖ZK和数据库，但都是弱依赖。ZK用于保存客户端的注册信息，如果ZK集群宕机，调度端会按之前缓存的客户端列表继续调度。数据库用于保存任务执行历史，如果数据库宕机，调度端仍然可以正常调度，但执行历史会丢失。
 轻量服务	用户不需要关心任务的调度逻辑，只需要实现业务逻辑即可。
 简单易用	用户只需要在业务应用里添加上Crane的客户端jar包，在需要调度的类上添加Crane的Annotation，然后在Crane管理端配置任务信息即可完成使用。

1.Crane是什么

Crane是一款任务调度中间件，支持分布式定时任务调度，支持国际通用的时间表达式（精确到秒），支持任务执行生命周期管理和历史执行记录查询，解决传统定时任务单点问题。可以用在比如

- 每秒钟进行一次视频转码
- 每小时清理一遍日志文件
- 每天进行的历史数据迁移

●每周五生成报表并发出邮件

● 每个月的最后一天发工资

● 每年一次的生日提醒等等场景

2.特性

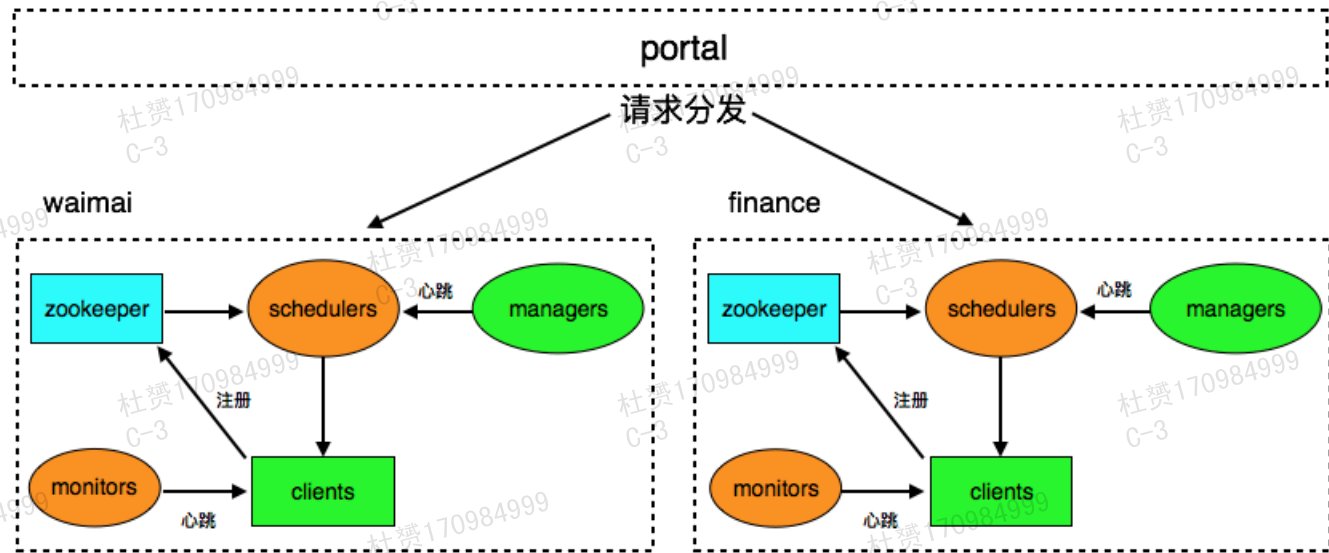
● 分布式：任务可以在多台客户端机器里的任何一台机器执行，如果客户端出现宕机的情况，调度端会自动选择存活客户端（业务机器）去执行任务。

● 高可用

● 轻量服务：用户不需要关心任务的调度逻辑，只需要实现业务逻辑即可。

● 简单易用：用户只需要在业务应用里添加上Crane的客户端jar包，在需要调度的类上添加Crane的Annotation，然后在Crane管理端配置任务信息即可完成使用。

整体架构



Crane在部署上根据BG进行隔离。

scheduler：调度器，负责对任务进行周期性调度

client：使用了crane的各个业务方的服务或者独立agent

manager：负责管理调度器集群，包括心跳维护、任务分配

monitor：负责管理client，对client进行心跳检测

用zk做注册中心，网络通信用netty，与RPC框架类似，scheduler相当于client，crane-client相当于server。

各模块解析

scheduler

调度器，作为crane业务使用方来说一般称之为crane服务端。

职责：负责对任务进行周期性调度。

架构


```
18 `retryInterval` int(11) NOT NULL DEFAULT '1',
19 `command` varchar(1024) DEFAULT NULL COMMENT '命令',
20 `hostName` varchar(1024) DEFAULT NULL COMMENT 'host name',
21 `poolID` int(11) NOT NULL DEFAULT '1' COMMENT '资源池ID',
22 `type` varchar(100) DEFAULT NULL COMMENT '任务类型',
23 `subType` varchar(20) NOT NULL DEFAULT 'Cron' COMMENT '子类型',
24 `fileName` varchar(200) DEFAULT NULL COMMENT '文件名',
25 `dockerInfo` varchar(4096) DEFAULT NULL COMMENT 'docker 镜像信息',
26 `description` varchar(1024) DEFAULT NULL,
27 `hadoopName` varchar(200) DEFAULT NULL COMMENT 'hadoop principle name',
28 `appName` varchar(200) DEFAULT NULL,
29 `isKillCongExp` tinyint(1) NOT NULL DEFAULT '0' COMMENT 'whether kill congestion expired attempt',
30 `isExpireDepTimeout` tinyint(1) NOT NULL DEFAULT '0',
31 `isNotConcurrency` tinyint(1) NOT NULL DEFAULT '1' COMMENT 'multiinstance',
32 `taskItem` varchar(1024) DEFAULT '' COMMENT 'data',
33 `taskClass` varchar(1024) DEFAULT NULL COMMENT 'class name',
34 `taskMethod` varchar(1024) DEFAULT NULL COMMENT 'method name',
35 `isOnePoint` tinyint(1) NOT NULL DEFAULT '1' COMMENT 'one node execute',
36 `isOneTime` tinyint(1) NOT NULL DEFAULT '0' COMMENT 'one time task',
37 `shardCount` int(11) NOT NULL DEFAULT '1' COMMENT 'shard count',
38 `taskBu` varchar(1024) DEFAULT NULL COMMENT '任务描述',
39 `location` varchar(20) DEFAULT NULL COMMENT 'location',
40 `taskGroup` varchar(64) DEFAULT '' COMMENT 'group',
41 `cell` varchar(150) DEFAULT NULL COMMENT '单元,逗号分隔',
42 `swimlane` varchar(150) DEFAULT NULL COMMENT '泳道指',
43 `checkAppKey` tinyint(1) DEFAULT '0' COMMENT '是否强制校验appKey信息',
44 `alarmTemplateId` int(10) DEFAULT NULL COMMENT '告警模板ID',
45 `auditTask` tinyint(1) NOT NULL DEFAULT '0' COMMENT '审计开关',
46 `markTest` tinyint(1) NOT NULL DEFAULT '0' COMMENT '是否压测流量',
47 `retryInterruption` tinyint(1) NOT NULL DEFAULT '0' COMMENT '中断自动重试',
48 `balanceStrategy` int(11) DEFAULT '0' COMMENT '客户端负载均衡策略',
49 `idc` varchar(64) DEFAULT NULL COMMENT '机房优先级',
50 `shardType` int(4) NOT NULL DEFAULT '0' COMMENT 'shard type',
51 `labels` varchar(1024) DEFAULT NULL,
52 `slot` int(11) DEFAULT NULL COMMENT '任务归属的slot',
53 PRIMARY KEY (`id`),
54 UNIQUE KEY `name` (`name`),
55 UNIQUE KEY `IX_taskID` (`taskID`)
56 ) ENGINE=InnoDB AUTO_INCREMENT=463 DEFAULT CHARSET=utf8;
```

可对应到界面的配置

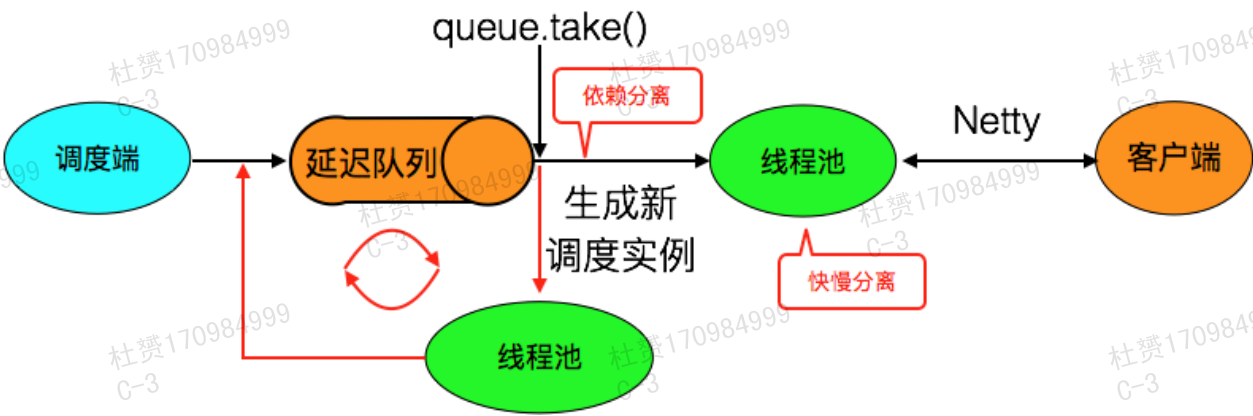


存储

用MySQL存储任务元数据

周期调度模型

采用原生Java提供的延迟队列，提前生成任务进入延迟队列，然后不断从队列获取到期的任务执行调度，同时再次生成任务进入队列，如此循环，驱动任务周期性调度。

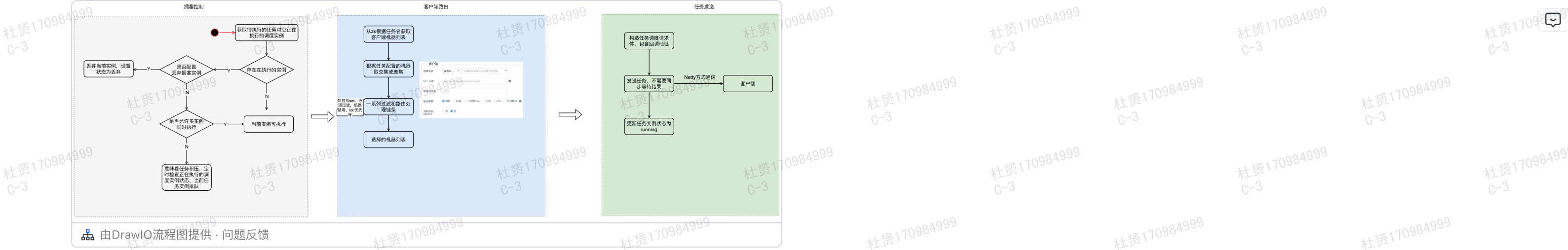


步骤如下：

1. 对于每个任务，**预先生成**下一次调度实例放入延迟队列，比如某个任务每个整点1小时执行1次，当前时间在10：30，则会设置延迟时间为半小时将调度实例放入队列。
2. 有多个线程以阻塞方式轮询该队列，即不断take()，拿出来的就是到期需要执行的任务
3. 执行任务调度实例，同时生成下一次任务实例重新放入队列

任务实例执行

1. 拥塞控制
2. 客户端路由
3. 任务发送



任务状态

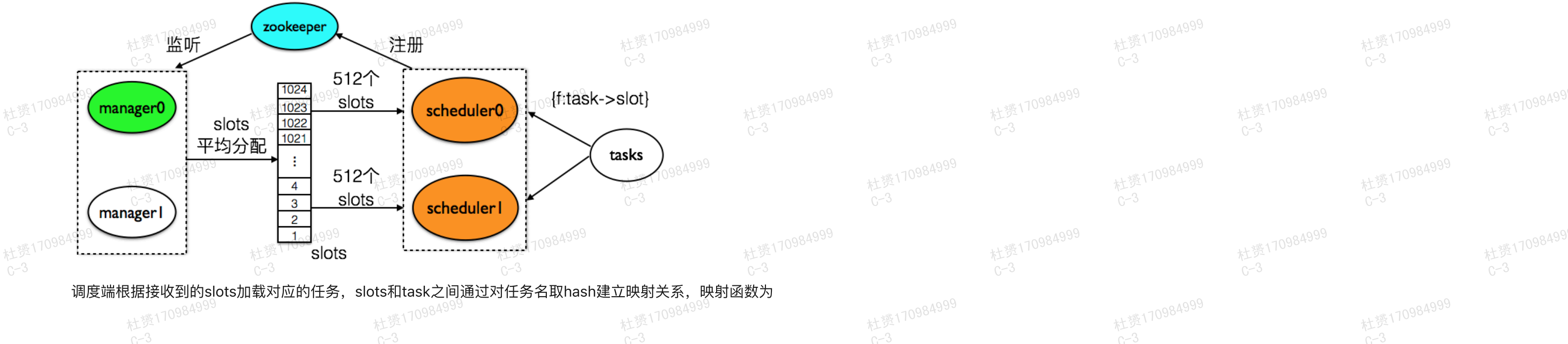


任务分配

为了应对任务数的不断增加，需要调度器具备水平扩展能力。所以，需要让每个scheduler只负责部分任务的调度。

实现方式

通过在任务和调度器之间增加一层slot，将任务散列到slot中，调度器负责不同的slot。



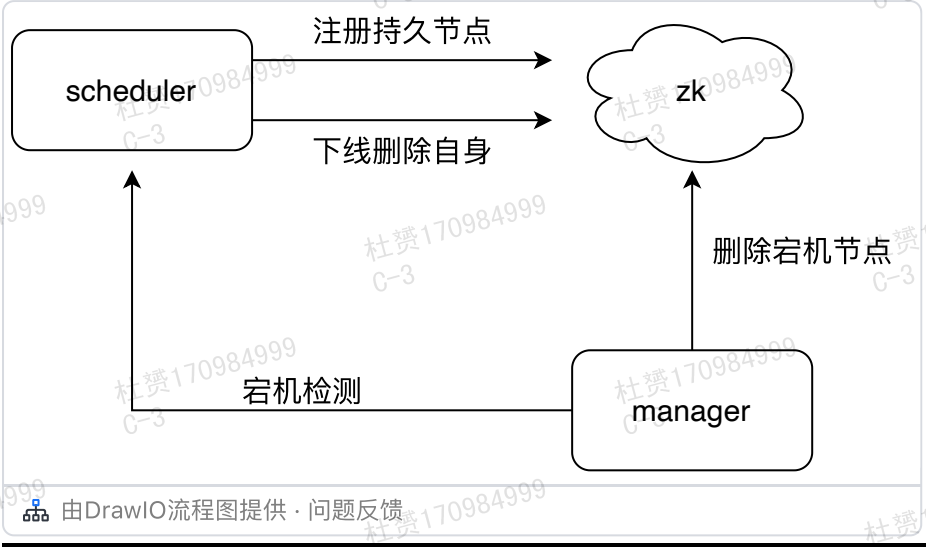
代码块

Plain Text

```
1 Math.abs(taskName.hashCode()) % 1024 + 1
```

具体slot如何分配给scheduler见下文manager模块。

scheduler上线与下线



scheduler注册

注册自身到zk持久节点，zk路径 `/crane/nodes/schedule/{ip}`, 注册后，由**manager发现scheduler**，进行slot分配，具体分配流程见下文manager模块。

scheduler下线

正常关闭时，主动从Zookeeper中删除自身节点，删除后，由**manager感知scheduler的下线**，进行slot重新分配。

异常情况宕机，无法主动从Zookeeper中删除自身节点，由manager通过对调度器进行**心跳检测**，从而摘除宕机节点，进行slot重新分配。

本地延迟队列的处理

- scheduler下线延迟队列中的任务信息就丢失了，但是由于任务是提前生成的，且会分配到其他scheduler处理，同样会提前生成然后到期分发，所以不会导致任务丢失不调度。
- 如果正好从延迟队列取出了任务后scheduler宕机，也就是此时任务应该要发送给客户端了但还没来得及发送，这种场景也会由**其他scheduler接管**，从DB获取到上次调度任务实例，然后根据配置生成任务调度实例并发送任务，可能产生调度延迟。

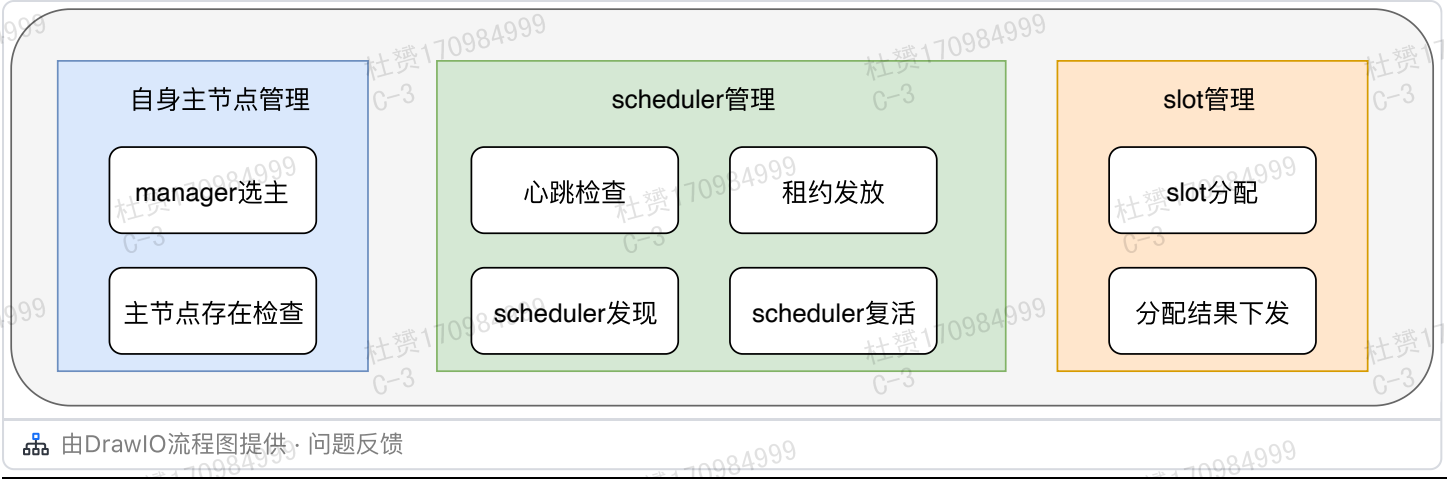
为什么不使用zk临时节点方式监控scheduler上下线？

发生网络抖动，某个scheduler节点无法连接到zk，超过session timeout时间zk server清除临时节点，而scheduler自己无法感知到，产生脑裂从而导致并发调度。

manager

职责：负责管理调度器集群，包括心跳维护、任务分配。负责用户在portal界面操作的响应。

架构



manager自身是有状态的，只有主节点才对scheduler进行租约发放、心跳检查以及节点上下线时的slot分配。

下面介绍其内部各功能的实现方式。

主节点管理

选主流程

manager启动时检查主节点信息，没有主节点则通过zookeeper选主，即**创建zk临时节点**，节点value为自身ip，**创建成功则作为主节点**，若当前已经存在节点则内存记录当前主节点ip。

只有自身是主节点才启动对scheduler的心跳管理。

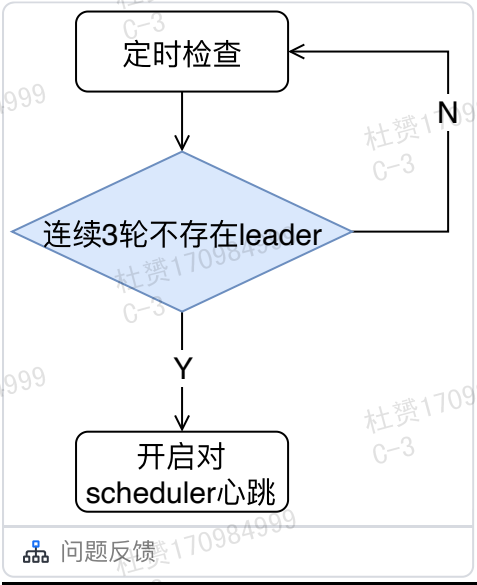
变更监听

每个manager注册了zk上manager主节点数据变更listener。当收到节点删除事件时，进入选主流程。

当收到节点数据变更时，若变更前自身为主节点变更后非主节点，则停止对scheduler的心跳维护，若变更前自身为非主节点变更后是主节点，则开启对scheduler的心跳维护。

主节点存在性检查

检查leader是否存在，目的是减轻对zk的依赖，**提高对zk的容错性**。



每个manager启动都会启动1个leader检查定时线程，每15s检查当前是否有leader，若连续3轮检查都没有leader，则开始心跳处理，避免zk宕机导致终止对scheduler发放租约。

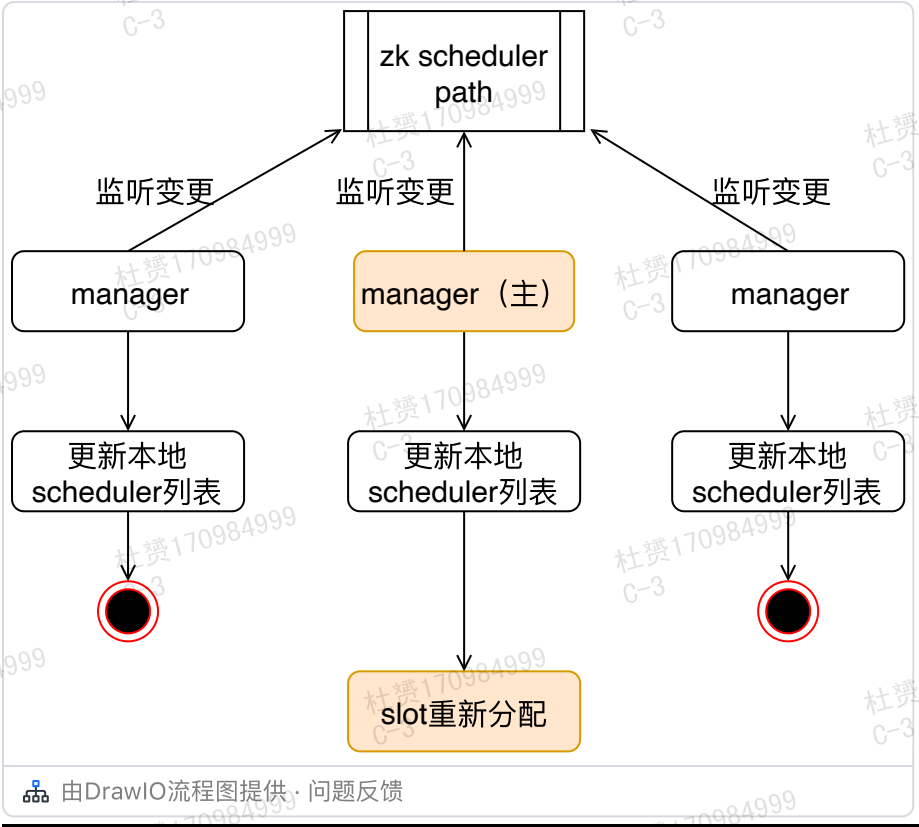
脑裂问题考虑

可以接受多个节点对scheduler进行心跳，保证scheduler不终止调度，同理，即使zk不可用，短时间内也能容忍。zk session超时时间配置6s。

scheduler管理

scheduler发现

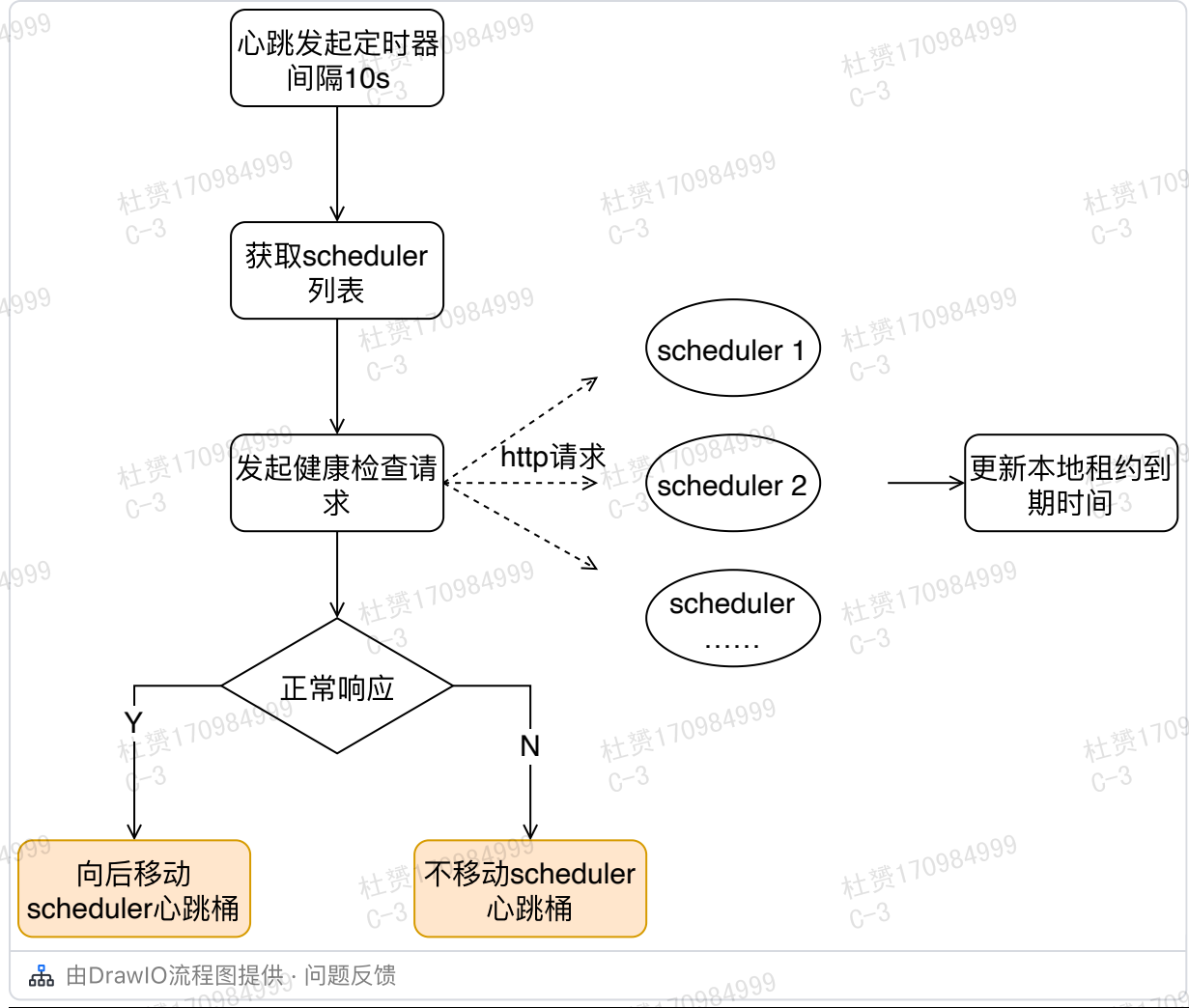
每个manager都注册zk scheduler节点变更listener，当收到节点增加、节点删除事件时，如果当前自身为主节点则进行slot重新分配。



节点删除比节点增加处理时多了一个步骤，如果有宕机的节点，则从心跳桶中移除，删除心跳相关信息

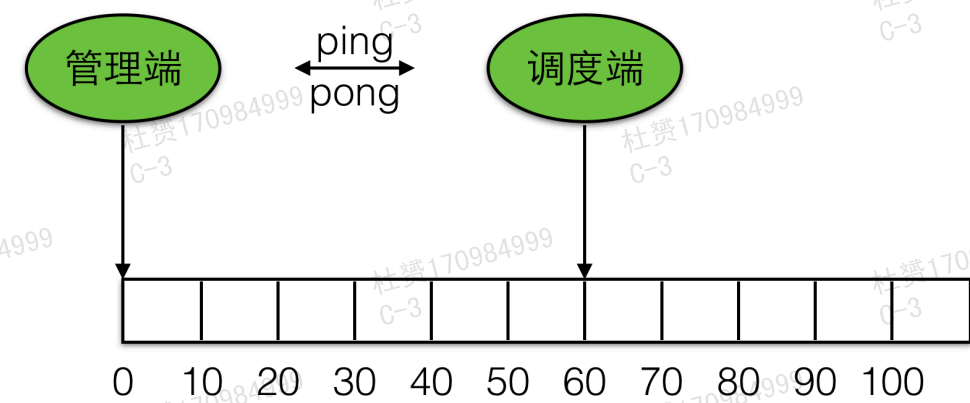
心跳检查与租约发放

采用基于租约的方式进行心跳管理，心跳检查同时发放租约，通过时间桶机制判断是否宕机。



每10s由manager的主节点发起对所有scheduler的心跳，通过http方式与scheduler通信。

时间桶机制



横坐标表示时间，0表示当前时间，每个桶的间隔是10秒钟，也就是每隔10秒钟ping一下调度端，调度端回复pong，则调度端往后移动到当前时刻+租约时长(60s)。

如果管理端和调度端一切正常，则管理端与调度端的时间间隔则一直为调度端的租期，此处为60秒钟。

时间桶检查

有一个单独的线程每隔10s检查一次当前时间戳的桶，时间桶有数据说明存在宕机节点。发现scheduler宕机时，从心跳检测桶里面移除，触发slot重新分配，加入到宕机内存缓存列表，更新内存中维护的scheduler列表。

宕机恢复

1个宕机scheduler检查线程。每10s检查内存缓存中保存的宕机节点列表，检查zk上如果存在该节点，发起健康检查请求，不存在则从内存缓存删除，检查成功也从内存缓存删除并重连，即触发slot重新分配，更新内存中维护的scheduler列表，检查失败则会继续检查，直到内存缓存过期后从zk删除节点，过期时长2h

slot管理

manager负责为每个scheduler分配Slots。

slot分配

分配算法

顺序平均分配。假设有m个scheduler，则每个scheduler负责的slot个数为 $\lceil \text{slot总数}/m \rceil$ 个，顺序将各个slot分配给scheduler，即如果slot为1024个，slot序号从1到slot总数/m 分配给第1个scheduler，从 $(\text{slot总数}/m)+1$ 到 $(\text{slot总数}/m)*2$ 分配给第2个scheduler，以此类推，剩余的再次顺序分配给各scheduler 1个。

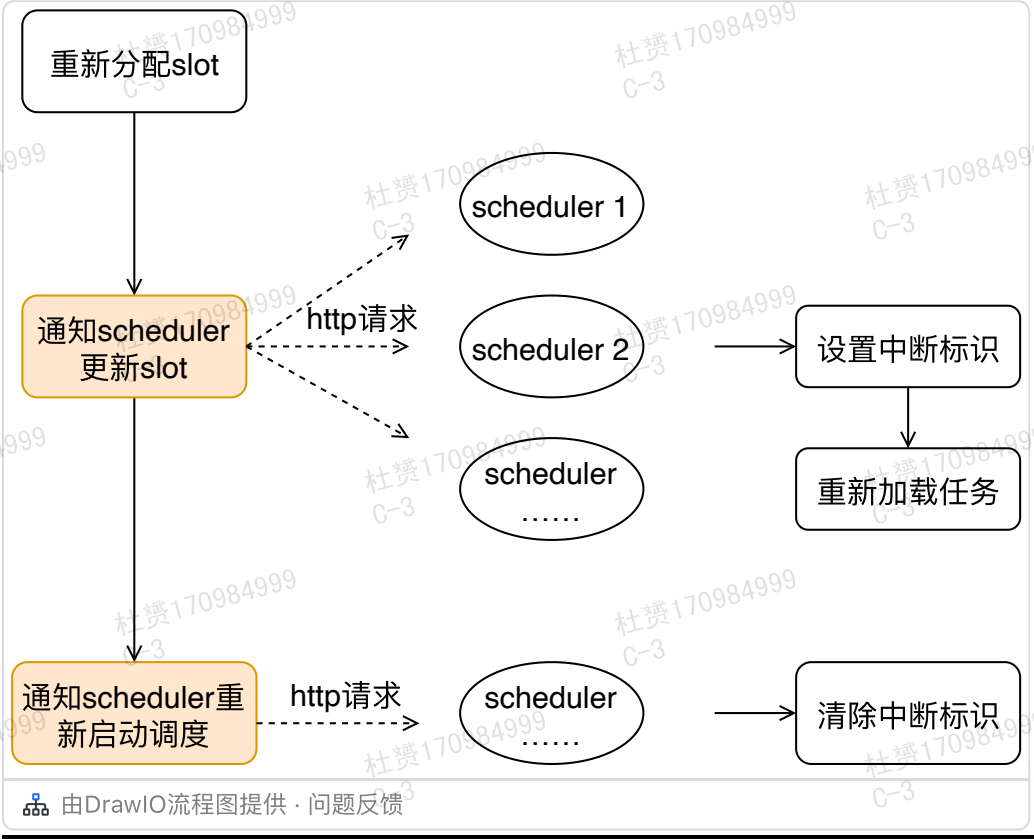
举例：

假设3个scheduler A、B、C，slot为1024个，则1-341的slot分配给A，342-682的slot分配给B，683-1023的slot分配给C，1024的slot分配给A。

重新分配

当发现scheduler上下线导致节点数量变更时，需要重新分配slots。

分配结果下发



通知scheduler重新启动调度的请求会尽可能等待所有通知scheduler更新slot的请求完成后再触发，实现上使用了CountDownLatch，但可以允许通知scheduler更新slot的请求失败。

slot信息同步

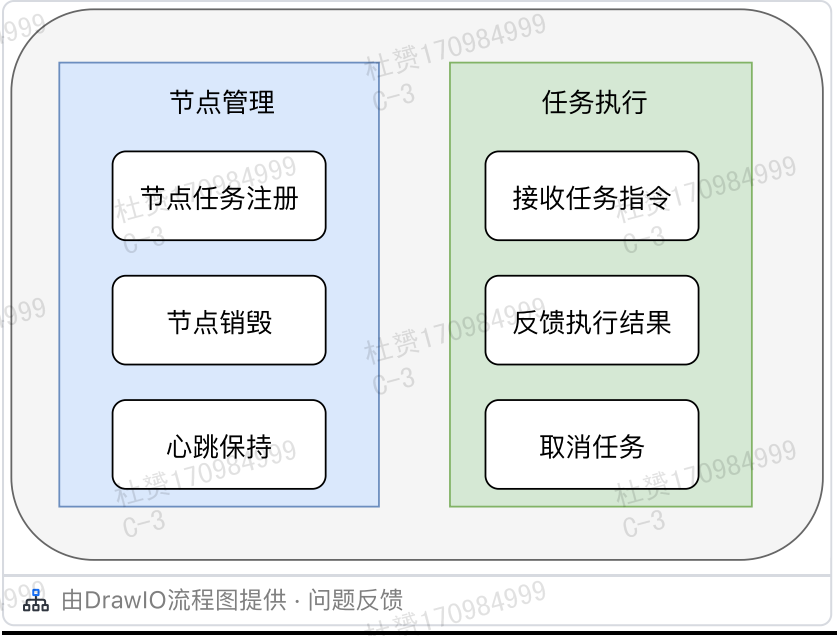
1个slot信息同步定时线程，每2mins从zk拉取scheduler的slot信息，更新到内存，维护scheduler对应的slot信息。

client

对于crane服务端而言，client就是使用了crane的各个业务方的服务或者独立agent。

职责：接收scheduler的调度任务，反馈任务执行状态

架构



节点注册

- 注册节点的job信息到zk持久节点，zk路径，`/crane/heartbeats/client/{addressAndPort}`，`value`内容 `{registeredJobs}`，注册后，`scheduler`在客户端路由时可通过zk获取到该节点，进行任务发送。
- 注册自身到zk持久节点，zk路径 `/crane/client/info/{addressAndPort}`，`value`内容 `{ClientInfo}`。

client初始化具体流程

- 根据spring xml配置或注解获取到需要调度的job
- 初始化注册中心配置、zk client
- 初始化各类指令对应的处理逻辑、回调逻辑

```
227     this.runningCallBacker = new AttemptRunningCallBacker();
228     this.runningCallBacker.setClient(client);
229     this.runningCallBacker.setPort(this.getTaskCallBackPort());
230
231     this.killCallBacker = new AttemptKillCallBacker();
232     this.killCallBacker.setClient(client);
233     this.killCallBacker.setPort(this.getTaskCallBackPort());
234
235     this.heartBeatCallBacker = new HeartBeatCallBacker();
236     this.heartBeatCallBacker.setClient(client);
237     this.heartBeatCallBacker.setPort(this.getTaskCallBackPort());
238
239     ScheduleProcessor scheduleProcessor = new ScheduleProcessor(taskCallBacker, coreSize);
240     scheduleProcessor.setApplicationContext(this.applicationContext);
241     scheduleProcessor.setMaxRunningSize(maxRunningSize);
242     LogRequestProcessor logRequestProcessor = new LogRequestProcessor(logCallBacker, serviceCapacity * 1);
243     AttemptRunningProcessor runningProcessor = new AttemptRunningProcessor(runningCallBacker, serviceCapacity * 1);
244     AttemptKillRequestProcessor killRequestProcessor = new AttemptKillRequestProcessor(killCallBacker, serviceCapacity * 1);
245     PingProcessor pingProcessor = new PingProcessor(heartBeatCallBacker, serviceCapacity * 1);
246     scheduleProcessor.registerAttemptListener(runningCallBacker);
247     OneTimeTaskScheduleProcessor oneTimeTaskScheduleProcessor = new OneTimeTaskScheduleProcessor(oneTimeTaskCallBacker, coreSize);
248
249     ExecutorService scheduleService = ThreadUtils.newThreadExecutor(coreSize, maximumPoolSize, coreSize <= 0 ? 1 : coreSize);
250     ExecutorService oneTimeTaskScheduleService = ThreadUtils.newThreadExecutor(maximumPoolSize * 8, maximumPoolSize * 8);
251     ExecutorService workService = ThreadUtils.newThreadExecutor(maximumPoolSize * 2, maximumPoolSize * 4, 10000);
252     ExecutorService attemptRunningWorkService = ThreadUtils.newThreadExecutor(maximumPoolSize * 4, 10000);
253     server.registerProcessor(CommandType.ScheduleSendTask, scheduleProcessor, scheduleService);
254     server.registerProcessor(CommandType.LogSendTask, logRequestProcessor, workService);
255     server.registerProcessor(CommandType.AttemptRunningSendTask, runningProcessor, attemptRunningWorkService);
256     server.registerProcessor(CommandType.AttemptKillSendTask, killRequestProcessor, workService);
257     server.registerProcessor(CommandType.HeartBeatSendTask, pingProcessor, workService);
258     server.registerProcessor(CommandType.ScheduleOneTimeSendTask, oneTimeTaskScheduleProcessor, oneTimeTaskScheduleService);
```

- 初始化NettyServer，对于scheduler发送任务给client而言client是server端
- 初始化NettyClient，反馈执行结果时请求scheduler
- 注册节点的job信息到zk
- 注册节点信息到zk，包括appkey, client版本，set，泳道信息
- 注册服务关闭的钩子

任务执行

任务接收与反馈通信方式



```
70     });
71
72     }
73
74     @PostConstruct
75     public void start() {
76
77         super.start();
78         if (callbackPort > 0) {
79             this.nettyServerConfig.setListenPort(callbackPort);
80         }
81
82         this.serverBootstrap.group(this.eventLoopGroup, this.eventLoopGroupWorker).channel(NioServerSocketChannel.class)
83             //
84             .option(ChannelOption.SO_BACKLOG, value: 1024)
85             //
86             .option(ChannelOption.SO_REUSEADDR, value: true)
87             //
88             .childOption(ChannelOption.TCP_NODELAY, value: true)
89             //
90             .childOption(ChannelOption.SO_SNDBUF, nettyServerConfig.getSocketSndbufSize())
91             //
92             .childOption(ChannelOption.SO_RCVBUF, nettyServerConfig.getSocketSndbufSize())
93
94             .localAddress(new InetSocketAddress(this.nettyServerConfig.getListenPort())).childHandler(new ChannelInitializer<NioServerSocketChannel>() {
95
96                 @Override
97                 protected void initChannel(NioServerSocketChannel ch) throws Exception {
98                     ch.pipeline().addLast(defaultEventExecutorGroup, //
99                         new NettyEncoder(), //编码
100                         new NettyDecoder(), //解码
101                         new NettyServerHandler(), //业务处理
102                         new NettyConnectManageHandler()); //异常处理
103                 }
104             });
105     }
```

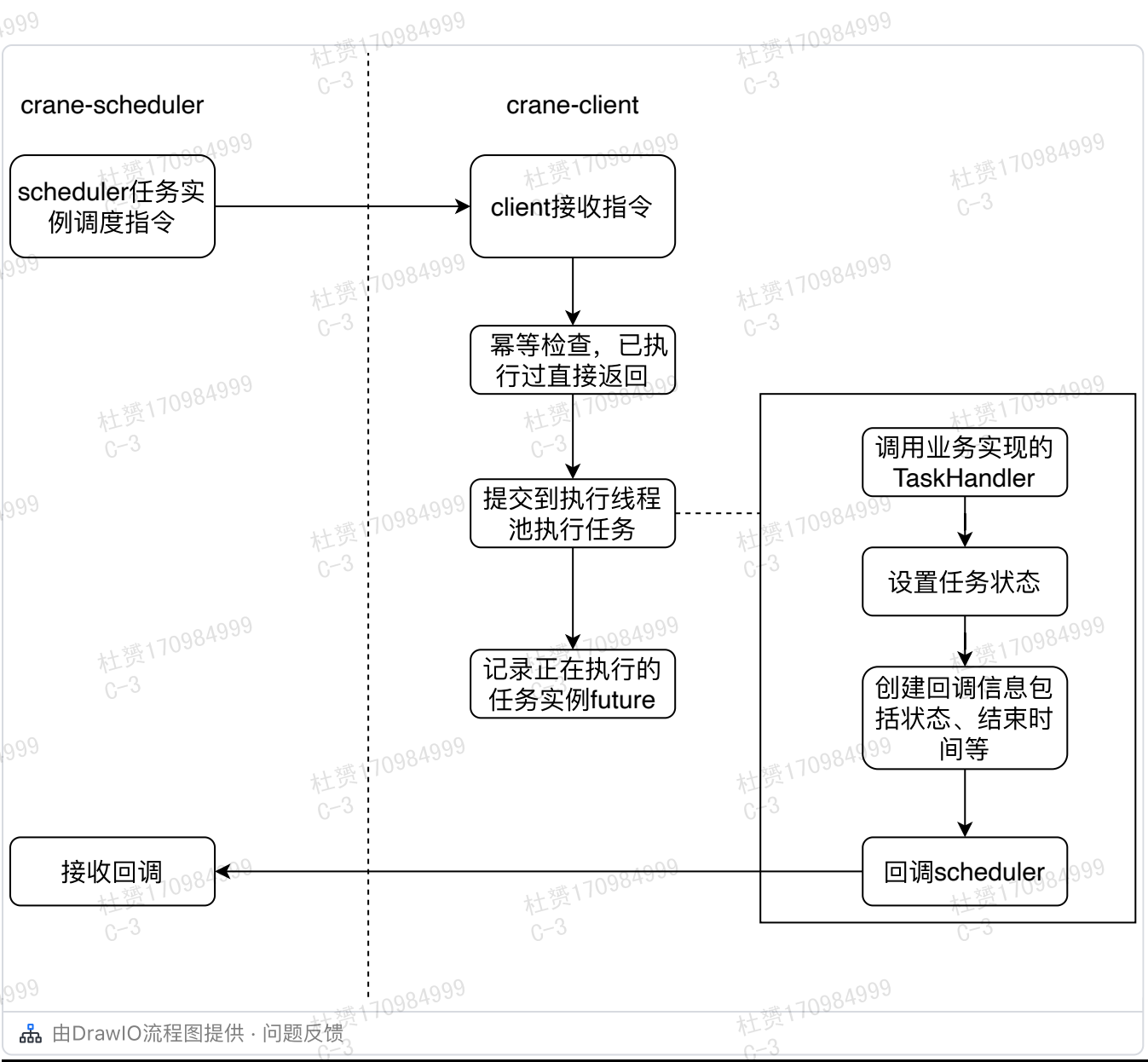
协议：4个字节的int表示数据的长度，接下来跟着的是把command转为json的bytes

任务执行流程

1. 收到scheduler发来的指令
2. 根据指令类型找到对应的处理器

```
public void processMessageReceived(final ChannelHandlerContext ctx, final Command msg) throws Exception {
    CommandType commandType = msg.getType();
    final Pair<NettyRequestProcessor, ExecutorService> pair = processorTable.get(commandType);
    if (pair != null) {
        Runnable run = new Runnable() {
            public void run() {
                try {
                    pair.getFirst().processRequest(ctx, msg);
                } catch (Throwable e) {
                    logger.error("process request exception", e);
                    logger.error(msg.toString());
                }
            }
        };
    }
    try {
        // 这里需要做流控，要求线程池对应的队列必须是有大小限制的
        pair.getSecond().submit(run);
    } catch (RejectedExecutionException e) {
        logger.warn(RemotingHelper.parseChannelRemoteAddr(ctx.channel()) //
            + ", too many requests and system thread pool busy, RejectedExecutionException: " //
            + pair.getSecond().toString() //
            + " request code: " + msg);
    }
}
```

以收到服务端发送任务实例为例，



NettyServer注册了多个processor，不同类型command对应不同的processor，不同的processor对应不同的处理线程池【线程池隔离】，processor处理业务逻辑，通过callback回调到server端。

心跳保持

接收心跳指令，回复pong，即指令执行成功，详见monitor部分。

服务关闭

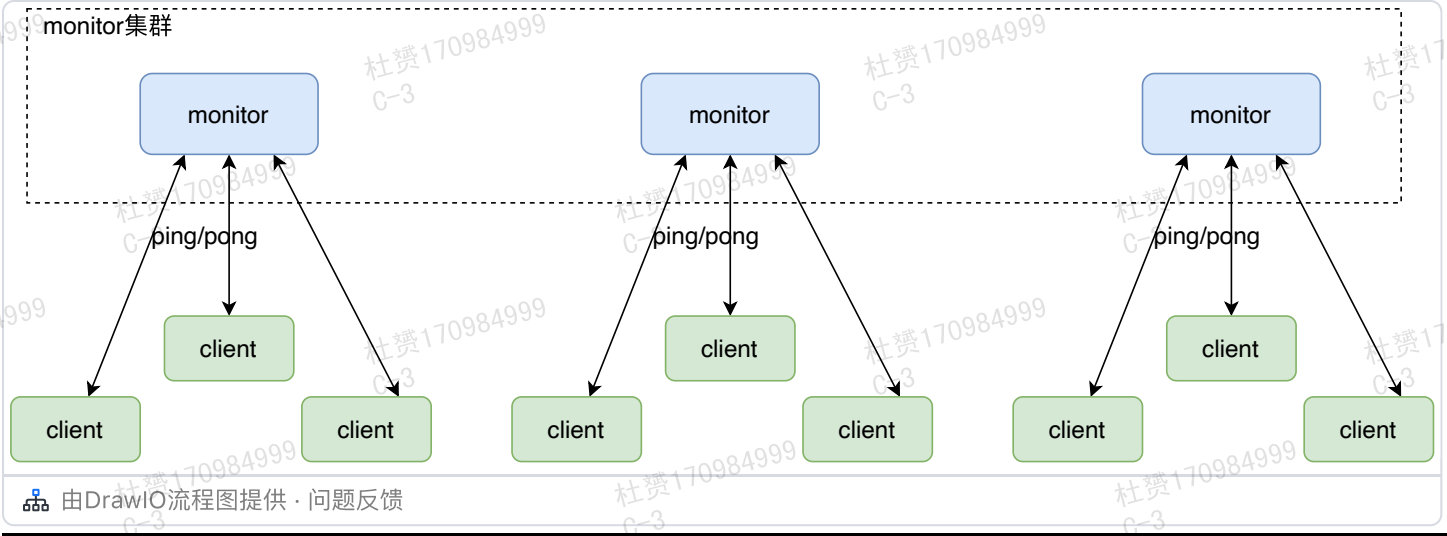
初始化时会注册JVM关闭的钩子，关闭服务会进行资源清理。包含如下步骤：

1. 删除注册在zk上的节点/crane/heartbeats/client/{自身ip:port}
2. 删除注册在zk上的节点信息 /crane/client/info/{addressAndPort}
3. 正在执行中的任务发送UNKNOWN的回调（可配置关闭前是否等待）
4. 关闭NettyServer，NettyClient

monitor

监控服务，负责管理client，对client进行心跳检测，摘除宕机的client。

架构

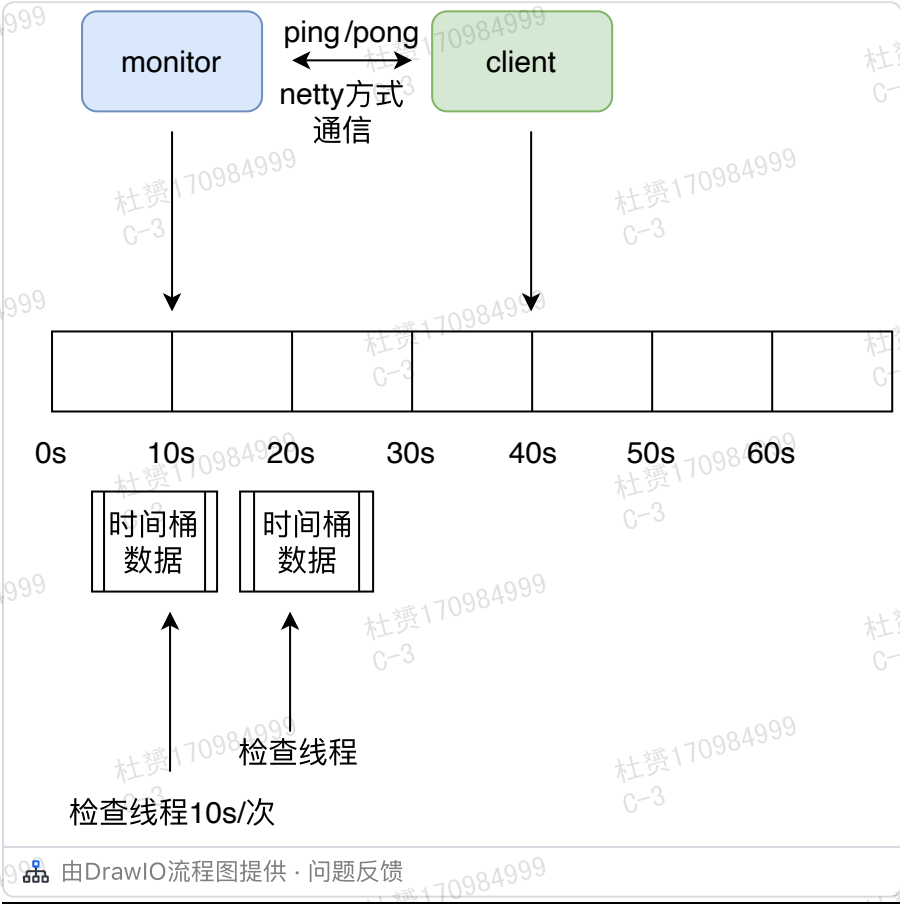


每个Monitor负责管理所有client中的一部分。

monitor自身注册zk持久节点/crane/heartbeats/monitor/{ip}, value为当前时间的前一个10s点, 定时10s一次更新时间戳。

租约机制

类似于manager与scheduler之间的心跳与租约机制, 采用时间桶的方式实现。



monitor发送心跳到client, 收到正常回复则时间桶移动到当前时刻+租约时长(30s), 分桶检测的线程不断移动检查该时间桶内是否有数据, 有则说明存在宕机节点。

心跳发送

启动单个定时线程池, 每10秒1次向自己monitor负责的client发送心跳。

具体流程:

1. 从zk获取所有client
2. 从zk获取所有monitor, 找到1分钟内有更新时间戳的monitor作为存活monitor, 同时也从zk上删除不存活的monitor节点

3. 若是自己负责的client，则通过健康检查线程池向client发送心跳请求

a. 分配client的方式：对monitor编号，用client的{ip:port} 的hashCode取模monitors数量，hash结果与monitor编号一致则属于该monitor负责。

4. 内存记录心跳请求历史

5. 通过netty发送请求到client

接收心跳的回调结果

收到正常回复则沿桶移动到当前时刻+租约时长(30s)，即删除旧桶数据，插入到下一个检查时间点的桶里。

时间点都是取整为下个10s的点。

宕机检查

启动1个分桶检测的线程，未到下个检查的时间点则wait，到时间点则获取到这个时间点对应桶内的数据，时间桶的数据也就是client，更新下次检查时间点，即当前时刻+10s。

如果桶里面有数据，说明在租约时间内没收到pong，设置心跳结果，从时间桶删除该client，若zk上存在该client节点且有心跳请求记录则判定为client宕机，删除心跳记录，zk上删除持久节点。

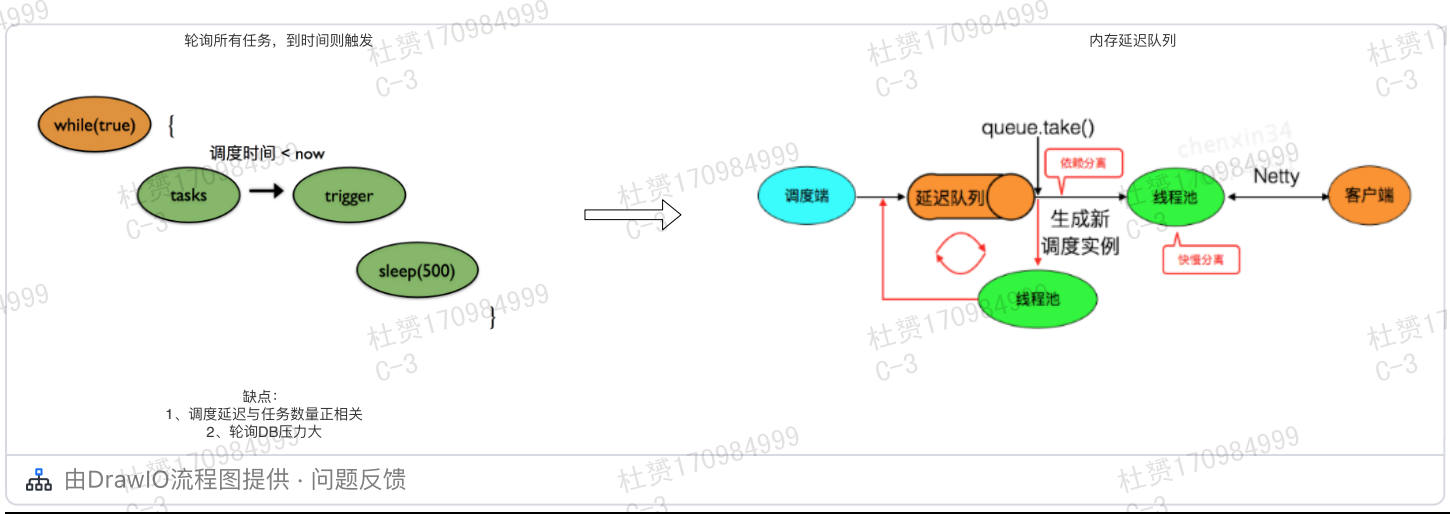
同时写入/crane/heartbeats/down/{ip:port} 持久节点，并放入内存缓存的宕机client列表中，继续检测。

单独的1个线程每隔5s检测宕机的client，也是发送心跳请求，如果收到pong，重新开始记录心跳，若zk没有该client节点信息，则重新写zk持久节点/crane/heartbeats/client/{ip:port}，删除内存缓存的宕机client列表，删除/crane/heartbeats/down/{ip:port}。

若宕机节点一直未恢复，则内存缓存2h后过期删除/crane/heartbeats/down/{ip:port}

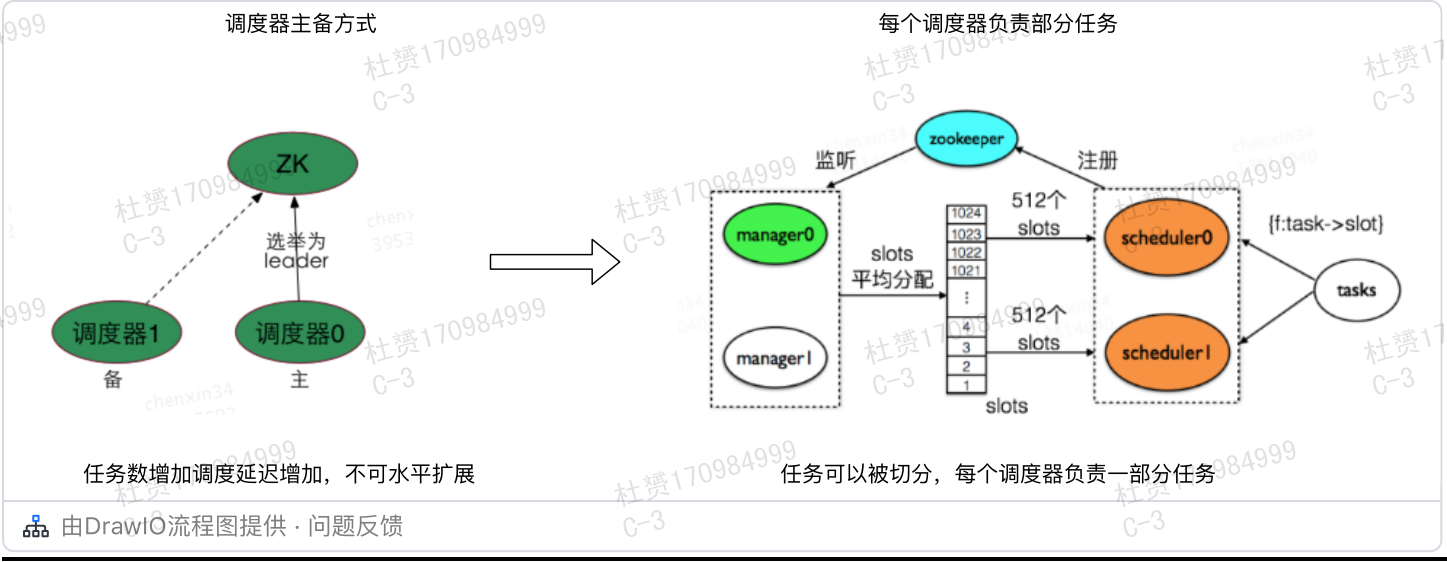
crane演进过程

定时任务触发模型



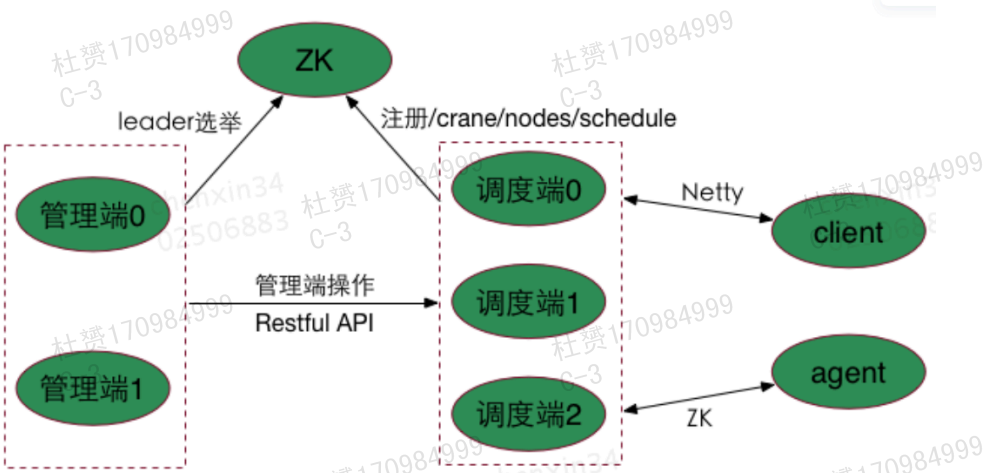
高可用改进

主备调度器演进为多台调度器并行



管理端和调度端分离

改进前管理端和调度端集成在一起作为一个整体



调度器在zk注册临时节点改为持久节点

zk临时节点在网络抖动、zk server抖动时可能造成临时节点被清除，带来slot的不断重新分配、脑裂问题，

参考

[Crane 基于lease高可用架构切换流程](#)

[crane任务状态](#)

[Crane 管理端和调度端HA解析](#)